

Machine Learning: cross-validation, decision trees, and random forests

Juan Pablo Lewinger

Recap: Lecture 1

Discussed 3 central issues in ML:

Evaluation bias: training performance always overestimates how well a model generalizes — for *any* model. Remedy: a held-out **test set**, used once only at the very end.

Overfitting: a model that is too complex fits noise rather than signal and performs worse on new data. Remedy: **control complexity** using a separate validation set to select the best model.

Model selection: we never know beforehand what model family (linear regression vs trees vs boosting) and which variables to include (including interactions and higher order terms). Want to try several candidate models with a range of complexity.

Together these motivate the three-way split: **train / validation / test**.

The problem with the three-way split

With $n = 252$ and a 60/20/20 split:

- **Training:** ~150 observations — to fit the model
- **Validation:** ~50 observations — to select the model
- **Test:** ~50 observations — to evaluate the model

With only 150 observations to fit the model we get a worse model than if we used more. 50 observations gives a noisy validation error — our model selection may be unreliable. And 50 observations gives a noisy test error.

The tension: we need data for three jobs, but splitting it three ways leaves too little for each.

This tension is especially painful in biomedical research — datasets with hundreds of patients are common; datasets with thousands are a luxury.

Today

- **Cross-validation** — a more efficient way to do model selection.
- **Decision trees** — a fundamentally different kind of model: flexible, interpretable, and naturally non-linear
- **Random forests** — fixing the key weakness of decision trees, at the cost of interpretability

Cross-validation: the key insight

In the three-way split, the validation set has two problems:

1. It's small — so validation error is noisy
2. Only a subset of all observations contribute to the validation error – we're at the mercy of which observations happened to end up in the validation set

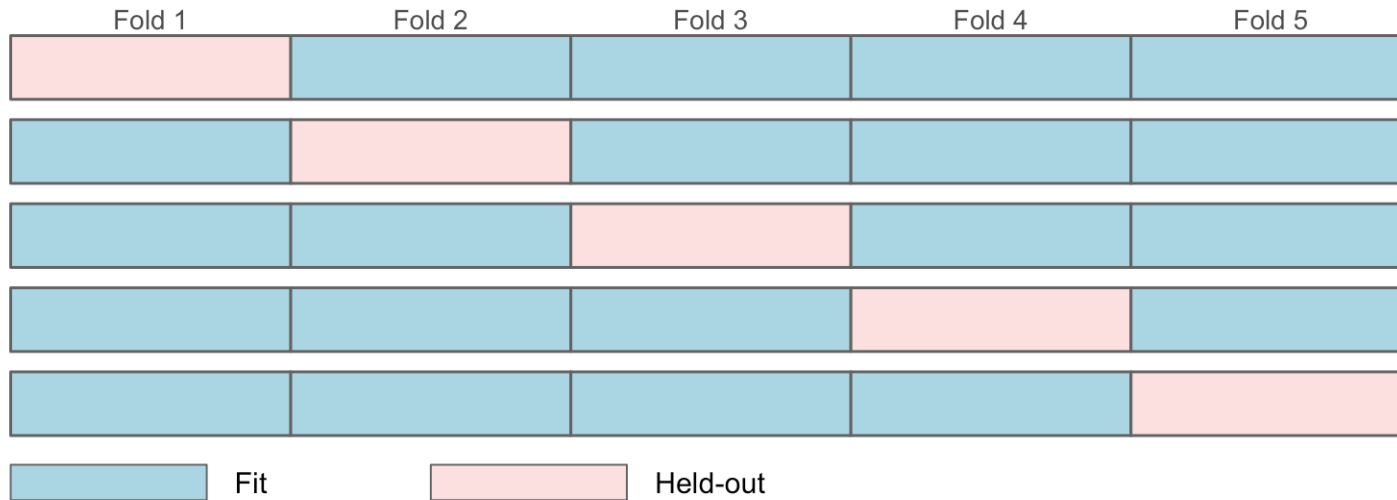
Key insight: instead of a fixed validation set, what if every observation got a turn at being held out for validation?

- Split the training data into k roughly equal **folds**
- For each fold in turn: fit the model on the other $k - 1$ folds, evaluate on this fold
- Average the k held-out errors

Every observation contributes to both fitting *and* evaluation.

K-fold cross-validation: the diagram

5-fold cross-validation on training data



What cross-validation gives you

After k iterations you have k held-out error estimates — one per fold. The **cross-validated error** is their average:

$$CV_k = \frac{1}{k} \sum_{i=1}^k \widehat{\text{Error}}_i$$

This single number estimates generalization error using *all* the training data. As an estimate of error, it has lower variability than one based on a single split.

$k = 5$ or $k = 10$ are standard in practice.

Cross-validation for model selection

Recall the problem with a fixed validation set: with only ~50 observations, validation error is noisy: our choice of model may depend heavily on which observations happened to end up there.

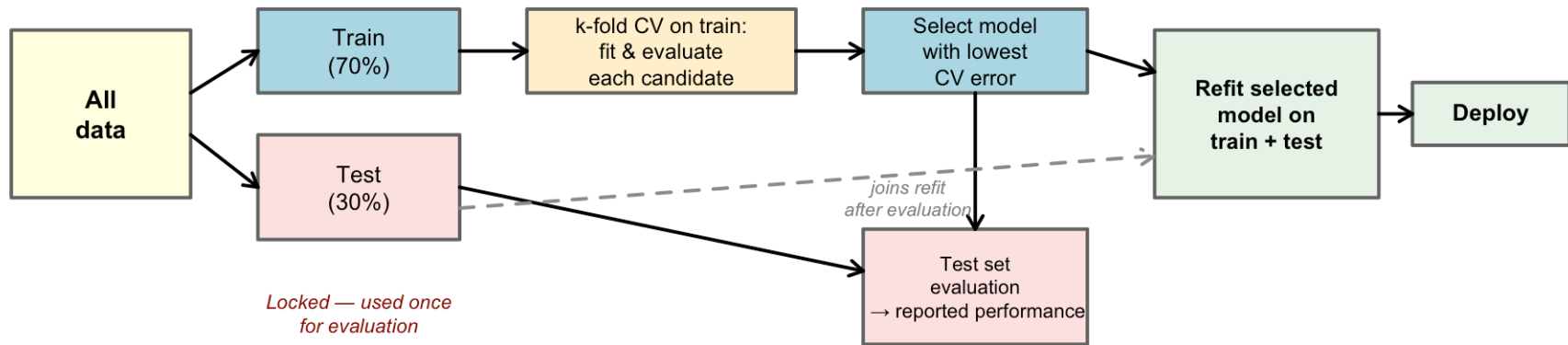
CV mitigates this: by rotating which fold is held out, every observation gets a turn in the validation role. The averaged CV error is a more stable estimate of generalization error, leading to more reliable model selection.

The workflow:

1. Set aside a **test set** at the very start — locked until the end
2. On the **training data**, use k -fold CV to estimate the generalization error of each candidate model
3. Select the model with the lowest CV error
4. Refit the selected model on the **full training set**
5. Evaluate once on the **test set** → this is your reported performance

The test set remains untouched until the end. CV does the job the validation set used to do — but more reliably.

Where CV fits in the workflow



Nested cross-validation

In the workflow we've seen, CV is used for model selection and the held-out test set gives the final performance estimate. This works well for most applications.

Nested CV adds an *outer* CV loop around the entire model selection process:

- **Inner loop:** CV on the training folds for model selection / tuning – just like before
- **Outer loop:** CV across the full dataset to estimate performance of the model selection procedure itself

Goal is to get a **lower-variance estimate of the final model's performance** than a single held-out test set provides. A single test set of 50 observations gives a noisy performance estimate; the outer loop averages over many such estimates.

Nested CV is computationally expensive and adds complexity.

For most biomedical applications with a reasonable sample size, the standard approach (CV for selection + held-out test for evaluation) is sufficient.

Nested CV is worth knowing about — especially when the dataset is too small to afford a held-out test set.

Beyond linear regression

So far we have used linear regression as our model throughout. It is powerful — but it has real limitations:

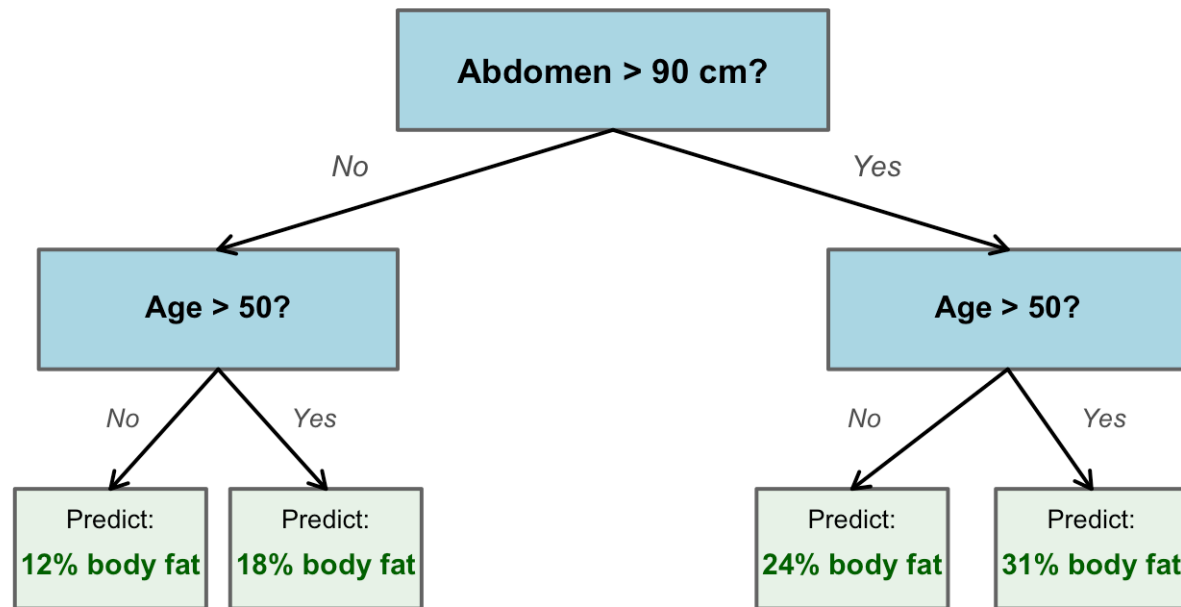
- Assumes a *linear* relationship between each predictor and the outcome
- Does not automatically capture interactions between predictors

What if the true relationship is non-linear? What if the effect of one predictor depends on the value of another?

We need more flexible models. Enter: **decision trees**.

Decision trees: the idea

A decision tree makes predictions by asking a sequence of yes/no questions about the predictors.



The prediction for a new patient is the average body fat % of training patients who reached the same leaf.

How splitting works

At each node the tree searches over **all predictors** and **all possible thresholds** to find the split that most reduces prediction error.

For regression trees (continuous outcome like body fat %):

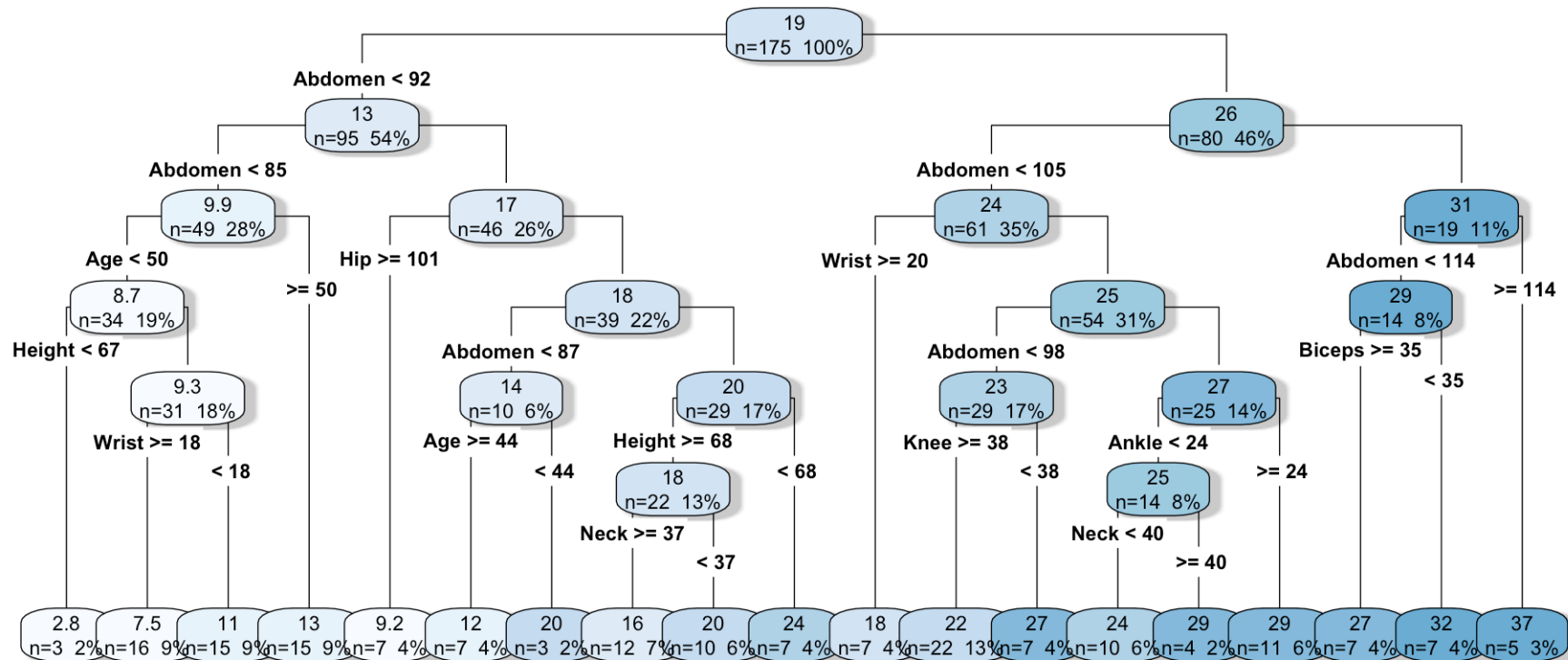
$$\text{Best split} = \arg \min_{j, t} \left[\sum_{i \in \text{left}(j, t)} (y_i - \bar{y}_{\text{left}})^2 + \sum_{i \in \text{right}(j, t)} (y_i - \bar{y}_{\text{right}})^2 \right]$$

Find the predictor j and threshold t that **minimizes the total sum of squared deviations** from the group means in the two resulting groups.

This is repeated at every node — the tree grows greedily, one split at a time.

A decision tree on the BodyFat data

Decision tree: BodyFat (full, unpruned)



Each node shows the predicted body fat % and the % of the 175 training observations reaching that node.

Reading the tree

- **Root node (top):** the first split — the single predictor and threshold that best divides the training data
- **Internal nodes:** subsequent splits on the subgroups
- **Leaf nodes (bottom):** the final prediction — the mean body fat % of training patients reaching that leaf
- **To predict a new patient:** follow the branches from root to leaf according to their measurements — the leaf value is the prediction

The tree is entirely transparent: you can trace exactly why any prediction was made. This is a major advantage over many other ML methods.

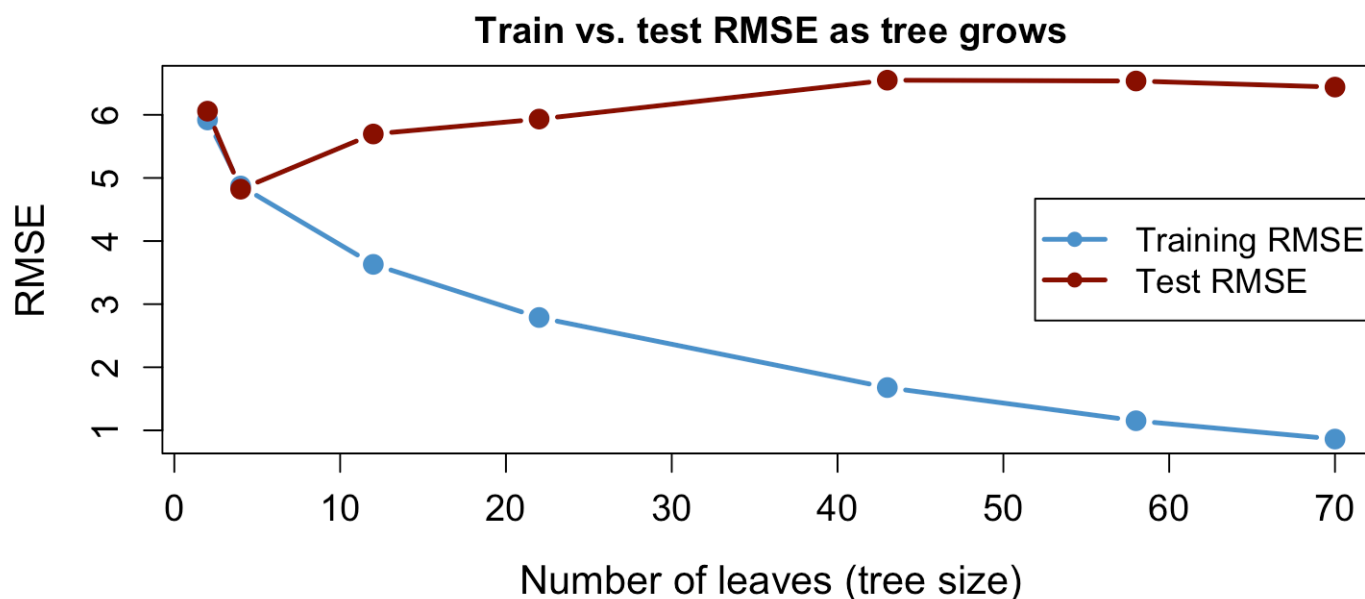
Strengths of decision trees

- **Interpretable:** the model is a flowchart — explainable to patients and clinicians
- **No data preprocessing needed:** handles mixed variable types, no need to scale predictors, tolerates missing data (with some implementations)
- **Naturally non-linear:** captures complex non-linear relationships and interactions between predictors without specifying them in advance
- **Fast to fit and predict**

The key weakness: overfitting

A deep tree that keeps splitting until every leaf contains just one or two patients will:

- Fit the training data almost perfectly
- Fail badly on new patients — it has memorized noise



The same overfitting pattern we saw with polynomials — training RMSE falls monotonically, test RMSE eventually rises.

Controlling tree complexity: pruning

Rather than growing a deep tree and accepting the overfitting, we **prune** — penalize complexity.

The fitted tree is chosen to minimize:

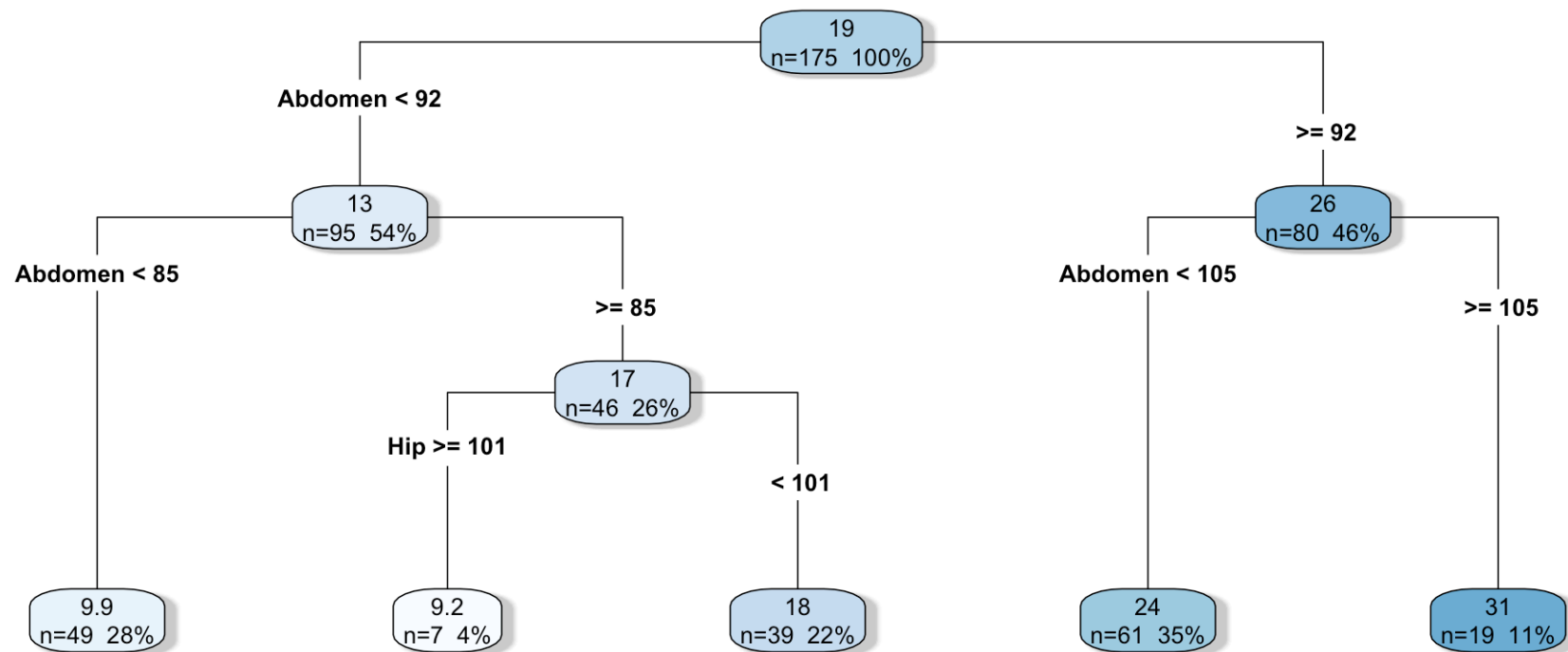
$$\sum_{\text{leaves}} \sum_{i \in \text{leaf}} (y_i - \bar{y}_{\text{leaf}})^2 + \alpha \cdot |\text{number of leaves}|$$

- The first term rewards fitting the data well
- The second term penalizes large trees
- $\alpha \geq 0$ controls the tradeoff — larger α = smaller tree

α (called **cp** in R's **rpart**) is a tuning parameter — chosen by cross-validation on the training data.

Pruned tree on the BodyFat data

Decision tree: BodyFat (pruned via 10-fold CV)



10-fold CV selected 5 leaves — simpler than the full tree, capturing the main patterns without memorizing noise.

Trees vs. linear regression

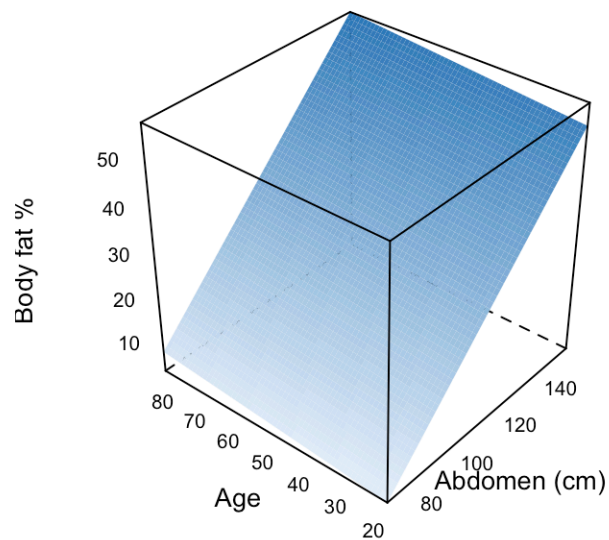
	Linear Regression	Decision Tree
Functional form assumed	Linear (a plane)	Piecewise constant in rectangular regions
Interactions	Must be specified explicitly	Captured automatically
Interpretability	Coefficients (direction + magnitude)	Visual flowchart
Overfitting risk	Low–moderate	High without pruning
Predictive accuracy	Good when relationship is truly linear	Good when relationship is complex; limited by high variance
Instability	Low	High — small data changes → very different tree

The instability row is crucial — it motivates random forests.

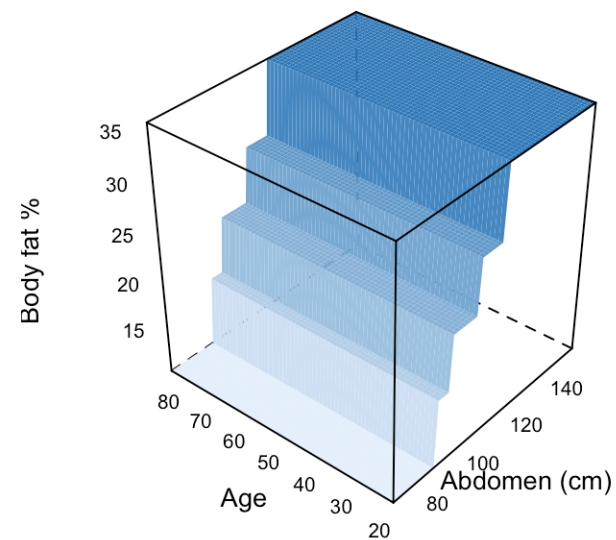
Prediction surfaces — 3D view

A tree predicts via a **piecewise constant function** in rectangular regions; linear regression predicts via a **plane**. Here we use two predictors (Abdomen, Age) to visualize this.

Linear regression — smooth plane

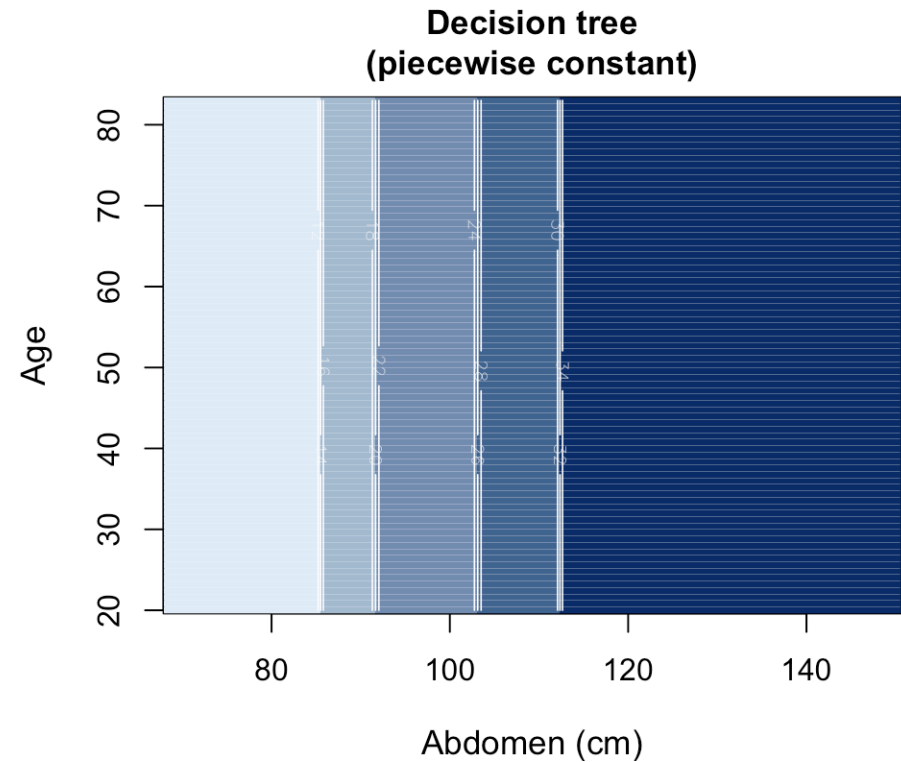
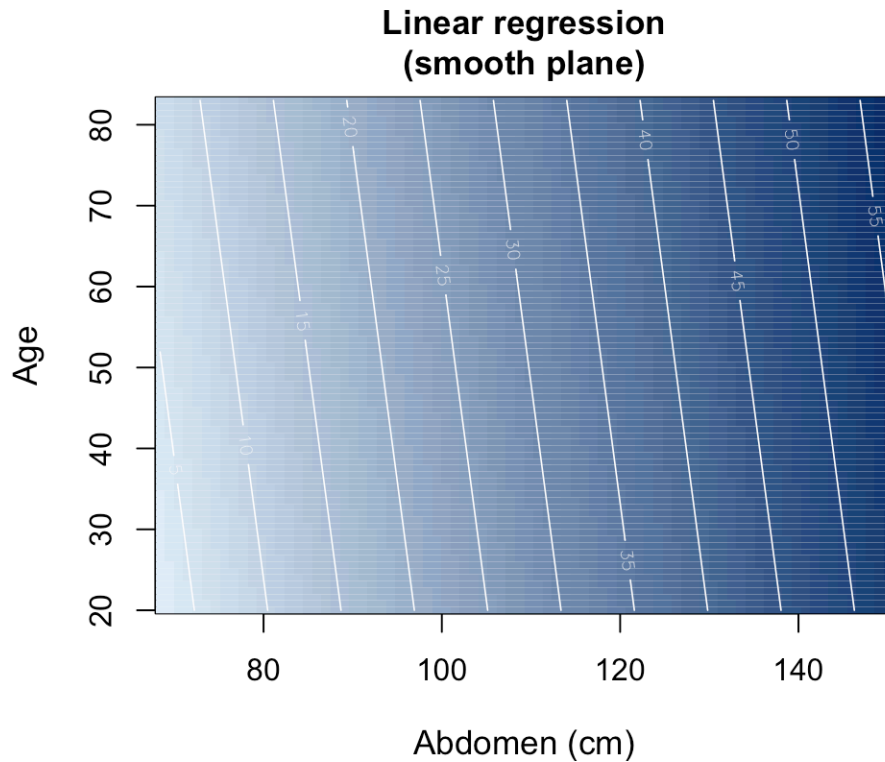


Decision tree — piecewise constant



The smooth tilted plane of linear regression vs. the staircase surface of the decision tree.

Prediction contours — tree vs. linear regression



The tree's rectangular regions are the decision boundaries from its splits. The linear model's smooth gradient reflects a single fitted plane.

The problem with a single tree: instability

Decision trees have high **variance**: small changes in the training data can produce very different trees.



Three different training sets → three very different trees. Which one do you trust?

The wisdom of crowds

If one noisy estimate is unreliable, what if we averaged many?

Analogy: ask one person to estimate the number of jellybeans in a jar — their guess may be way off. Ask 500 people — the average will be more accurate. Individual errors cancel out.

The same principle applies to models:

- A single tree is unstable and prone to overfitting
- The average of many trees is stable and generalizes well
- Individual trees are wrong in different ways — their errors cancel

This is the core idea behind **random forests**.

The bootstrap: sampling with replacement

A **bootstrap sample** is a random sample drawn *with replacement* from the original data, the same size as the original.

Bootstrap resampling: 8 original observations → 3 bootstrap samples

:	1	2	3	4	5	6	7	8
:	1	5	1	1	2	4	2	2
:	1	8	7	8	7	4	1	5
:	6	4	2	2	7	3	1	1

Red dash = not selected in this sample. Each sample has ~63% unique observations.

On average each bootstrap sample contains ~63% unique observations; the rest are duplicates.

Building a random forest: two sources of randomness

A random forest builds many trees, each deliberately made different through two injections of randomness:

1. Bootstrap sampling (bagging)

Each tree is fitted on a *bootstrap sample* — a random sample of the training data, drawn *with replacement*, the same size as the original.

- On average, each bootstrap sample contains ~63% of the training observations
- The remaining ~37% (“out-of-bag” observations) can be used to estimate error

2. Random feature subsets

At each split, only a *random subset* of predictors is considered — not all of them.

- Typical choice: $\sqrt{p}$ predictors for classification, $p/3$ for regression
- This forces diversity among trees — even with the same bootstrap sample, trees diverge

Why random feature subsets?

Without random feature subsets, all trees would tend to use the same strong predictors at the top (e.g. abdomen circumference for body fat). The trees would be highly correlated — and averaging correlated estimates doesn't reduce variance much.

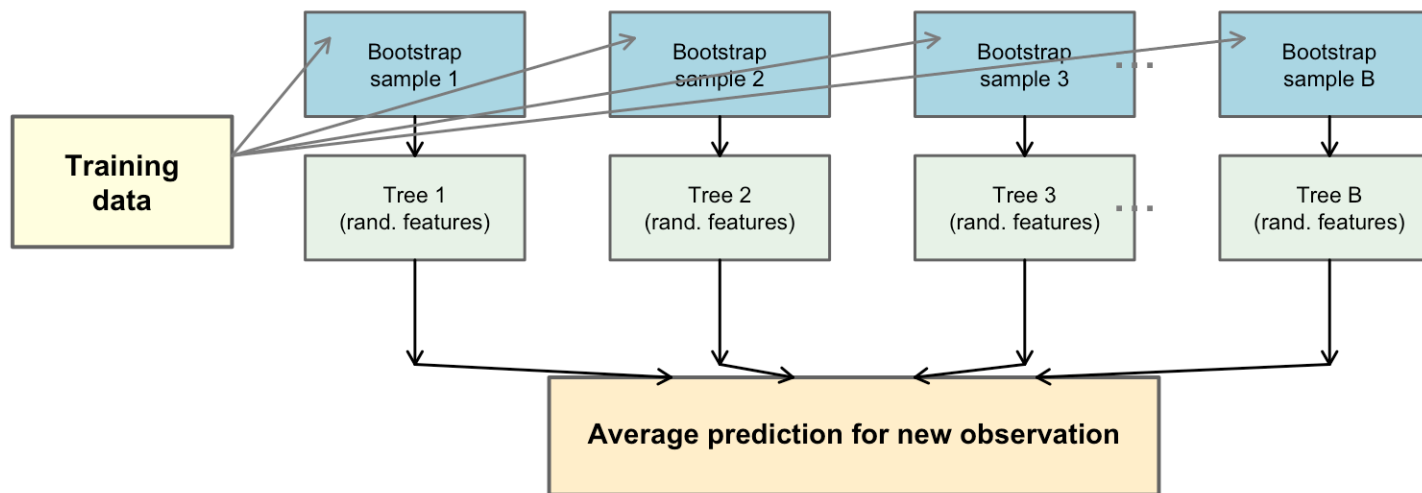
By randomly restricting which predictors each split can use:

- Different trees use different predictors
- Trees become less correlated with each other
- Averaging them produces a much larger reduction in variance

This is the key insight that makes random forests more than just “bagging with trees.”

The random forest algorithm

Random forest: B trees $\rightarrow$ average prediction for a new observation



To predict a new observation: run it through all B trees, obtain B predictions, take their mean.

What random forests gain and lose

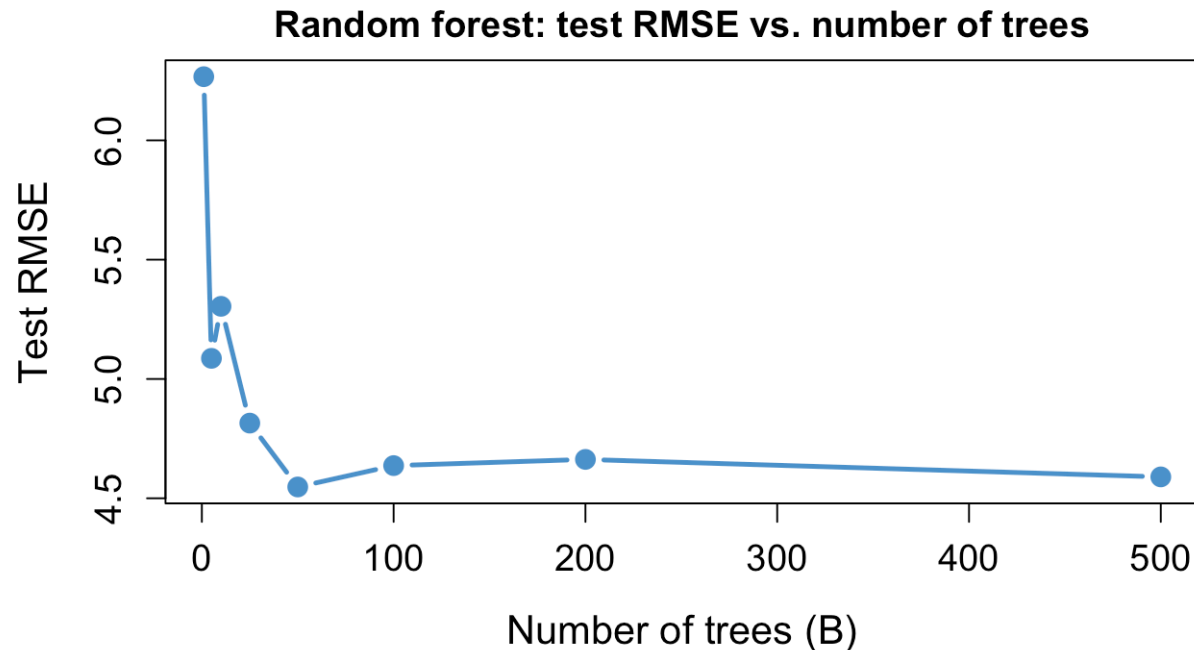
Compared to a single decision tree:

	Single Tree	Random Forest
Predictive accuracy	Moderate	Much better
Stability	Low — sensitive to data changes	High
Overfitting	Severe without pruning	Much reduced
Interpretability	High — visual flowchart	Low — can't draw the forest
Tuning needed	Pruning parameter α	Number of trees B , features per split

The fundamental tradeoff: **accuracy vs. interpretability**. In clinical settings, both matter — and the right choice depends on the application.

How many trees?

Unlike the polynomial degree or pruning parameter, the number of trees B is not a typical model selection problem — more trees always helps (or at worst doesn't hurt), so there is no overfitting as B increases.

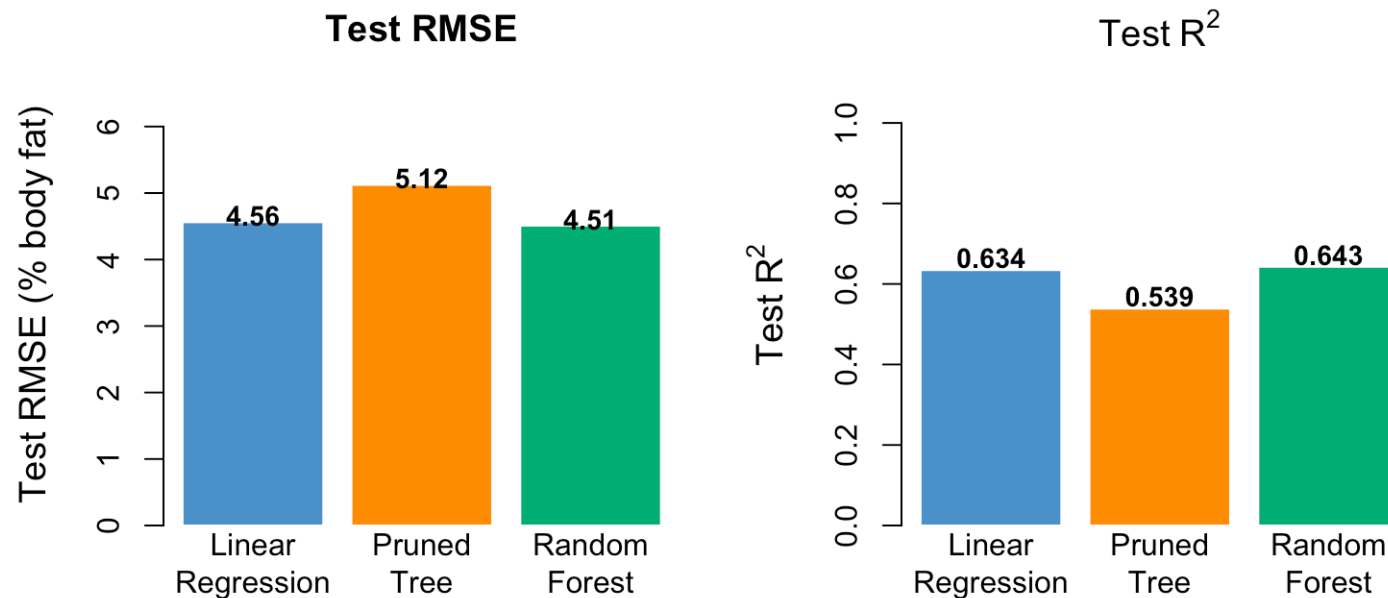


Error stabilizes well before $B = 500$. In practice, $B = 500$ is a safe default.

Putting it all together: model comparison

We compare three models on the BodyFat data, all fitted on the same training set (70%, all features except **Density**) and evaluated on the same test set (30%):

- **Linear regression:** all circumference and demographic predictors (linear terms only)
- **Pruned decision tree:** same predictors, pruned via 10-fold CV
- **Random forest:** same predictors, $B = 500$ trees

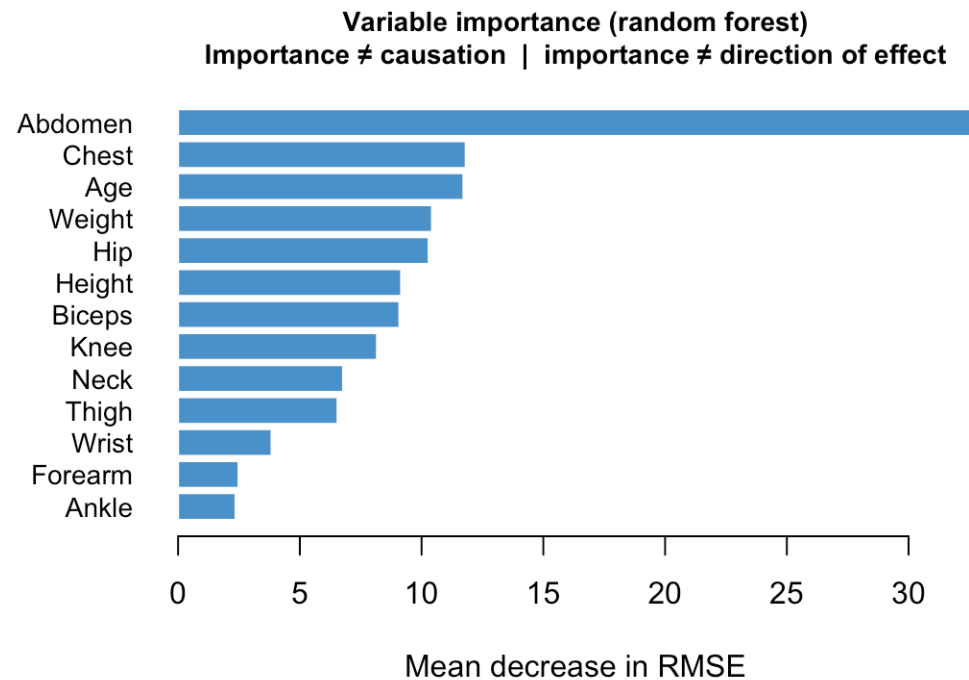


All three evaluated on the **same held-out test set** — a fair comparison.

Variable importance

A random forest can't be drawn — but we can ask: **which predictors contributed most to reducing prediction error?**

How it's computed: for each predictor, the total reduction in squared error across all splits on that predictor is averaged over all B trees — predictors that drive large, frequent reductions rank highest.



Key takeaways

- **Cross-validation** replaces the fixed validation set with a rotating one — using all training data for both fitting and model selection, giving more reliable estimates with less waste
- **CV for model selection + held-out test for final evaluation** is the standard rigorous workflow
- **Decision trees** are flexible, interpretable, and naturally capture non-linear relationships — but a single tree overfits and is unstable
- **Random forests** fix both problems by averaging many diverse trees — much better accuracy and stability, at the cost of interpretability
- **The accuracy-interpretability tradeoff** is fundamental in ML. In clinical settings, both matter — the right choice depends on how the model will be used and by whom

Ethics: the black box problem in medicine

Random forests predict well — but a clinician cannot look inside the model to understand *why* it made a particular recommendation.

This raises real questions in clinical ML:

- **Accountability:** Accountability: if a model recommends a treatment and the patient is harmed, can you explain why that recommendation was made? With a black-box model, you cannot.
- **Trust:** clinicians are more likely to act on a recommendation they can understand
- **Regulation:** the FDA increasingly requires explainability for AI-based medical devices
- **Bias:** a black-box model may be discriminating on protected attributes without anyone noticing

The field of **explainable AI (XAI)** is actively working on tools to peer inside complex models — but no method yet matches the transparency of a linear model or a single tree.

This doesn't mean black-box models shouldn't be used in medicine — it means their use requires extra care, validation, and governance.

Where to go from here

More ML methods worth knowing:

- **Regularized regression** (Lasso, Ridge): linear models that handle many predictors — great for genomics
- **Gradient boosting** (XGBoost, LightGBM): sequential tree building — often the best performer in practice
- **Neural networks / deep learning**: powerful for images, text, and genomic sequences

Learning resources:

- *Introduction to Statistical Learning* (James et al.) — free PDF at statlearning.com — the clearest textbook introduction, with R code
- *Tidy Modeling with R* (Kuhn & Silge) — free at tmwr.org — modern R workflow using **tidymodels**
- Coursera: *Machine Learning Specialization* (Andrew Ng) — great conceptual foundation

In R: the **tidymodels** ecosystem provides a unified framework for the entire workflow — CV, model fitting, tuning, and evaluation — across all model types.

Final thoughts

ML is a powerful set of tools — but it is not magic.

- A model is only as good as the data it was trained on
- Evaluation discipline (train/CV/test) is what separates a trustworthy model from an optimistic one
- Interpretability is not always possible — but it should always be *considered*
- The goal is not the most sophisticated model; it is the model that best serves the patient

You now understand the core ideas that underpin almost everything in modern ML. The methods get more complex — the principles stay the same.